KPFL Week 1 Requirements and Design

KPFL Week 1 Requirements and Design....1
1. Requirements....3
Functional requirements....3
Non-functional requirements....3
6-Week Project Schedule....4
2. Actors, constraints and business rules....5
Actors....5
Constraints....5
Mandatory business rule....5
3. Use cases....6
Register and approve a team....6
Create a fixture....6
Assign a match official....6
Record a result and update standings....7
Sell tickets and shop products....7
4. Design decisions....8
Changes from the recommended model....9
5. Persistence and validation strategy....10
Files....10
How saving and loading work....10
Validation strategy....11
6. Initial test plan....11
7. UML Diagram....15
8. Console Menu Design....16
Main Menu....16
1. Team Management Submenu....16
2. Fixtures & Match Operations Submenu....16
3. Ticketing & Shop Submenu....16
4. Management Reports Submenu....17
TO DO

1.Finalize Team Roles....17
2.Lecturer Confirmation of the minimum player age constant....17
Project Repository:....18
Repository Visibility:....18
Main Branch:....18

1. Requirements

The system has 12 groups of functional requirements, one for each core requirement in the brief, and 6 non-functional requirements.

Functional requirements

1. Register, update, search and display football teams, including name, parish, registration number, squad size and league status.

2: Add, update and remove players and coaches on a team.

3: Register stadiums with ID, name, location, seating capacity, field condition and ticket price.

4: Create fixtures by assigning two teams, a stadium, a date and a time.

5: Register match officials and assign them to fixtures without scheduling conflicts.

6: Record match results, calculate league points (win 3, draw 1, loss 0) and produce sorted standings.

7: Sell tickets by category and monitor stadium capacity.

8: Manage shop products (merchandise, food and beverages) and stock, and complete shop transactions.

9: Calculate subtotal, tax, total payment and change, and produce a receipt

10: Save all data to files and reload it when the program starts.

11: Produce at least three management reports: league standings, fixtures with attendance and ticket revenue, and shop sales and low stock.

12: Reject invalid data and prohibited operations using custom exceptions with a clear message.

Non-functional requirements

1: The program is menu driven. An invalid entry shows a clear message and asks again instead of crashing.

2: A missing or damaged data file is reported, and the program still starts. Saved data is never overwritten with partial data.

3: Money is calculated so that tax and change are exact, not rounded by floating-point errors.

4: Code is split into model, exception, service, storage and user-interface packages, one class per file.

5: Business rules live in the model and service classes, not in the menu, so they can be tested without the console.

6: All sources and AI-assisted work are acknowledged as the course policy requires.

6-Week Project Schedule

Week	Focus	Key Deliverables
Week 1	Requirements & Design	Functional & non-functional requirements, use cases, domain model, class diagram.
Week 2	Domain Model Implementation	Model classes, encapsulation, business rules validation, initial unit tests.
Week 3	Persistence & File I/O	Sequential file storage (teams, people, fixtures, stadiums) and random access file handling (products, tickets).
Week 4	Services & Business Logic	League and shop service operations, custom exceptions handling, report generation logic.
Week 5	User Interface & Testing	Console menu system, input validation, execution of full test plan with results and evidence recorded.
Week 6	Review, Refinement & Final Submission	Code cleanup, final documentation review, comprehensive testing, project submission.

2. Actors, constraints and business rules

Four actors use the system, and eight mandatory business rules from the brief are each enforced by a named class and a custom exception.

Actors

Actor	Role in the system	Main goals
League Commissioner	Runs the league and is the main operator.	Approve and suspend teams, register stadiums and officials, create fixtures, assign officials, record results, view reports.
Coach or team representative	Looks after one team's squad.	Add, update and remove players and coaches.
Spectator	Attends a match.	Buy tickets and shop products.
Shop attendant	Operates the stadium shop (assumed, because the brief names no shop operator).	Record sales, take payment, give change and receipts, update stock.

Constraints

- Object-oriented design is marked first. A GUI may complement the solution but cannot replace the design, validation, exceptions or tests.
- Data is stored in sequential files, random access files, or both.
- Shop prices include 15% GCT, as the scenario states.
- The brief mentions a league minimum player age but never gives it. It will be one constant in one place until the lecturer confirms the value.

Mandatory business rule

- An unregistered or suspended team cannot be assigned to a fixture.
- A team cannot play two fixtures at the same day and time.
- A match official cannot be assigned to conflicting fixtures.
- Ticket sales cannot exceed the stadium's seating capacity.
- Match scores and quantities cannot be negative.
- Players must meet the league minimum age.
- A transaction cannot be completed when payment is insufficient.
- A fixture cannot be completed until a valid result is recorded.

3. Use cases

Five use cases cover teams, fixtures, officials, tickets and the shop, and between them they exercise all eight business rules.

Use case	Primary actor
Register and approve a team.	Coach, Commissioner
Create a fixture.	Commissioner
Assign a match official.	Commissioner
Record a result and update standings	Commissioner
Sell tickets and shop products.	Spectator, Shop attendant

Register and approve a team.

Actors: Coach (registers), Commissioner (approves). **Precondition:** none.

1. The coach enters the team name, parish, registration number and squad size.
2. The system checks the data and stores the team with the status 'Pending'.
3. The coach adds players; the system checks each player's age (BR6).
4. The Commissioner selects the pending team and approves it. The status becomes registered.

Create a fixture

Actor: Commissioner. **Precondition:** two Registered teams and one stadium exist.

1. The commissioner chooses the home team, away team, stadium, date and time.
2. The system checks both teams are registered (BR1) and are different teams.
3. The system checks that neither team has a fixture at that date and time (BR2).
4. The fixture is stored with status Scheduled.

Assign a match official

Actor: Commissioner. **Precondition:** a scheduled fixture and an available match official exist.

1. The Commissioner selects a fixture and an official.
2. The system compares the fixture's date and time with every fixture the official already holds (BR3).
3. The official is added to the fixture.

Record a result and update standings

Actor: Commissioner. **Precondition:** the fixture is scheduled.

1. The Commissioner selects the fixture and enters the home and away goals.
2. The system checks that both scores are whole numbers of zero or more.
3. The system stores the result and marks the fixture as completed.
4. The system awards points (win 3, draw 1, loss 0) and rebuilds the standings.

Sell tickets and shop products.

Actors: Spectator, Shop attendant. **Precondition:** a scheduled fixture and products in stock exist.

Tickets

1. The spectator picks a fixture, a ticket category and a quantity.
2. The system checks that the quantity is above zero (BR5).
3. The system checks tickets already sold, plus the quantity does not exceed the seating capacity (BR4).
4. The tickets are issued at the stadium ticket price.

Shop

5. The attendant adds products and quantities for the spectator, and the system checks that each quantity is above zero (BR5) and that stock is enough.
6. The system works out the total, the GCT and the subtotal.
7. The spectator pays. The system checks the payment covers the total (BR7), works out the change and prints a receipt.

4. Design decisions

Inheritance is used only where "is a" holds, composition only where the part cannot outlive the whole, and everything else is aggregation or plain association.

Relationship	Type	Why
Person -> Player, Coach, Official, League Commissioner, Spectator	Inheritance	All five share id, name, age and gender, and each adds its own data.
Product -> Merchandise, Food Item	Inheritance	One shop inventory holds both, and they differ only in extra attributes.
Football Team & Player	Aggregation	A player can leave a team and join another, so the team does not own the player.
Football Team & Coach	Aggregation	A coach is assigned to a team but exists independently of it.
Football League -> Football Team, Stadium, Match Official	Aggregation	They register with the league but are independent entities.
StadiumShop & Product	Aggregation	Products exist in the catalogue regardless of any one sale.
Football League & Fixture	Composition	A fixture has no meaning outside its league.
Fixture & MatchResult	Composition	The result is created when the fixture is completed and has no life of its own.
Fixture & Ticket	Composition	A ticket is for one match, and capacity is counted per fixture.
Fixture & Football Team	Association	A fixture refers to a home and an away team, and neither owns the other.
Fixture & Stadium	Association	A stadium hosts many fixtures over time.
Fixture & Match Official	Association	An official works many fixtures, and a fixture has several officials.
Spectator & Ticket	Association	Each ticket is bought by one spectator.

Changes from the recommended model

The brief allows groups to change its recommended model when the change is consistent, documented and justified. The group made these three changes.

Brief's model	Our Design	Why
Coach stores a team assignment	Coach has no team field. FootballTeam holds its list of coaches	The team already aggregates its coaches, so a coach's team is found through the team. Storing it in both classes would duplicate the link, and the two copies could disagree.
Stadium stores occupancy and the teams assigned to the current fixture	Occupancy is counted per fixture from its tickets (Fixture.getOccupancy() and Fixture.getSeatsRemaining()), and the teams come from the fixture	One stadium hosts many fixtures, and each fixture sells its own tickets, so occupancy belongs to the fixture (Fixture & Ticket composition). Storing it in Stadium would duplicate data that changes with every sale and between matches
Stadium ticket price (UML shows `double`)	BigDecimal	Non-functional requirement 3: money must be exact, with no floating-point rounding errors

5. Persistence and validation strategy

League and people data are saved in sequential text files, and products and tickets in random access files, so the project shows both approaches the brief allows. If time runs short, the two random access files can be changed to sequential without touching the model.

Files

All files live in `data/`. Text fields are separated by `|` because names and locations may contain commas.

File	Access	Holds	Key
`teams.txt`	Sequential	registration number, name, parish, squad size, status	registration number
`people.txt`	Sequential	id, name, age, gender, role, role-specific fields, and the team's registration number for players and coaches	person id
`stadiums.txt`	Sequential	id, name, location, capacity, field condition, ticket price	stadium id
`fixtures.txt`	Sequential	id, date, time, team registration numbers, stadium id, status, goals, official ids	fixture id
`products.dat`	Random access, fixed-length records	id, type, name, price in whole cents, stock, extra field	record number from product id
`tickets.dat`	Random access, fixed-length records	ticket id, fixture id, spectator id, category, price in whole cents	record number from ticket id

How saving and loading work

- Records store ids, not objects. After loading, a second pass links the objects together, so files load in this order: stadiums, teams, people, fixtures, products, tickets.
- League standings are not stored. They are rebuilt from completed fixtures after every load and every result.
- Data loads when the program starts and saves on exit and from a menu option.
- A save writes to a temporary file first and then replaces the old file, so a failed save never destroys the previous data.
- A missing file means first run and loads nothing. A damaged record raises `DataFileException` naming the file and line, and nothing from that file is loaded.
- Every loaded record goes through the same constructors as typed input, so an edited file cannot bypass a rule.

Validation strategy

Each rule is checked in one layer, and every check runs before any data changes, so a refused operation leaves no partial update.

Layer	What it checks
Menu	The text is a number, date or time and is not empty.
Model constructors and setters	Single-field rules: blank names, negative scores, quantities and prices, minimum player age, duplicate IDs.
League and shop methods	Rules involving several objects
Storage	Every loaded record is checked again.

6. Initial test plan

There are 28 planned test cases covering normal boundary and invalid scenarios for the core business rules as well as system persistence and error handling. Each business rule has a normal and an invalid case and every rule except BR8 also has a boundary case, since BR8 is a state rule with no numeric limit.

Approach

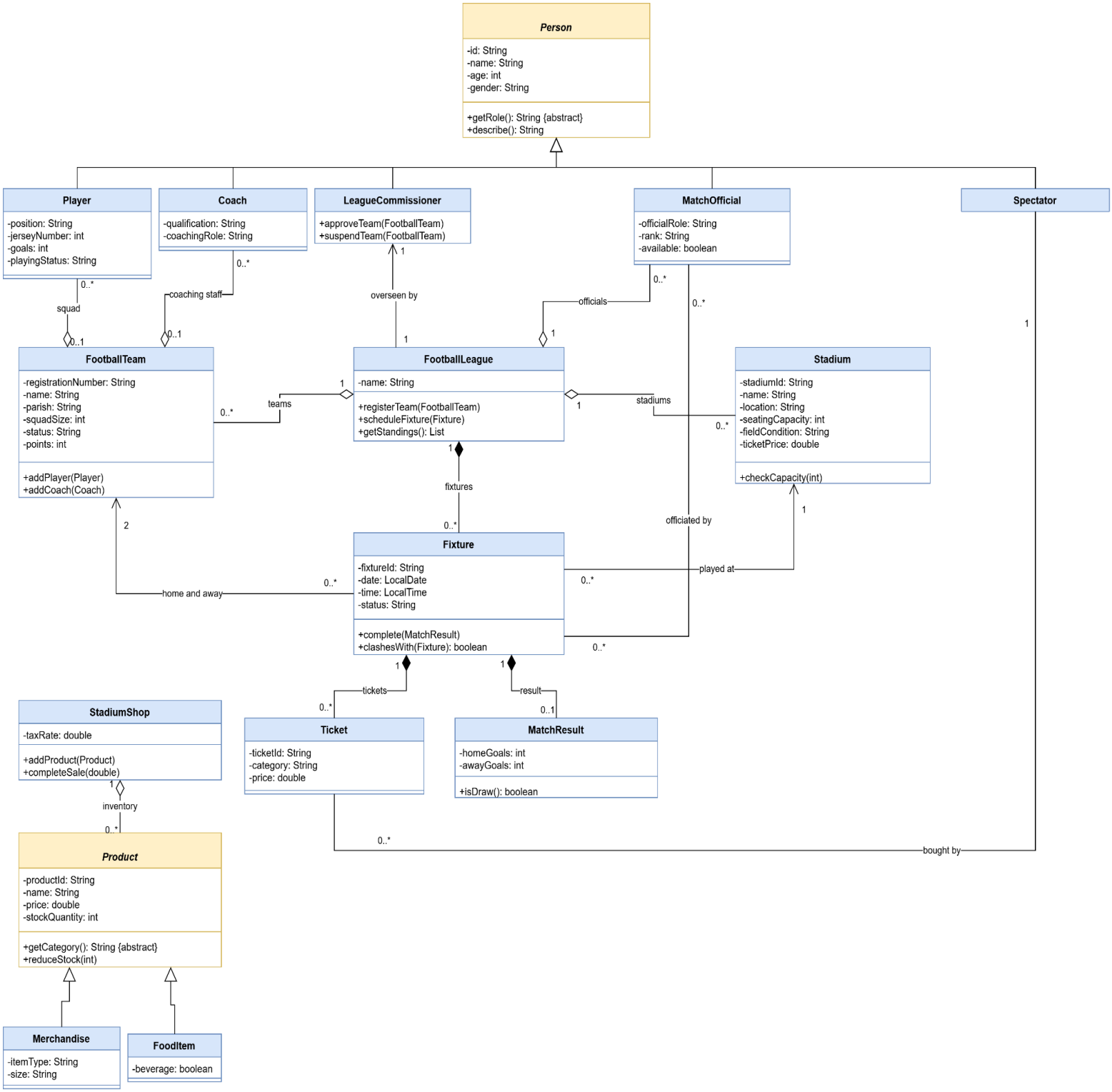
- Model and service rules are tested with unit tests so they run without the console.
- Menu behaviour is tested with a written keystroke script, with a console log kept as evidence.
- Tests start from the same sample data files so results are repeatable.
- Result and Evidence columns are added in Week 5.
- `MIN` is the league minimum player age and `C` is a stadium's seating capacity.

Type	Setup and input	Expected	Result	Evidence
Normal	Two Registered teams and a free stadium.	Fixture stored as Scheduled.		
Boundary	A team approved a moment earlier (Pending to Registered).	Accepted immediately.		
Invalid	Pending or Suspended team used in a fixture.	TeamNotEligibleException; no fixture stored.		
Normal	Team has a fixture at 15:00; second	Accepted.		

Type	Setup and input	Expected	Result	Evidence
	fixture the same day at 18:00.			
Boundary	Second fixture the same day at 15:01.	Accepted; only an identical date and time clashes.		
Invalid	Team already has a fixture at that date and time, as home in one and away in the other.	SchedulingConflictException.		
Normal	Official assigned to fixtures on two different days.	Both assignments stored.		
Boundary	Official assigned to two fixtures the same day at different times.	Accepted.		
Invalid	Official assigned to two fixtures at the same date and time.	SchedulingConflictException; official's fixtures unchanged.		
Normal	0 sold; sell 10.	10 sold.		
Boundary	Sold plus quantity equals C.	Accepted; 0 seats left.		
Boundary	Sold equals C; sell 1.	StadiumCapacityExceededException.		
Invalid	Sold plus quantity equals C + 1.	StadiumCapacityExceededException; sold count unchanged.		
Normal	Result 2 to 1.	Result stored.		
Boundary	Result 0 to 0; ticket quantity 1.	Draw accepted; quantity accepted.		

Type	Setup and input	Expected	Result	Evidence
Invalid	Score of -1; ticket quantity 0 and -1; shop quantity -1.	InvalidValueException each time; nothing stored.		
Invalid	Score typed as letters.	Menu asks again; the model is never called.		
Normal	Player 5 years above MIN.	Accepted.		
Boundary	Player age equals MIN.	Accepted.		
Boundary	Player age equals MIN - 1.	UnderagePlayerException.		
Normal	Total 2,645.00, paid 3,000.00.	Change 355.00; stock reduced.		
Boundary	Paid equals total.	Accepted; change 0.00.		
Boundary	Paid is 0.01 below total.	InsufficientPaymentException; stock unchanged.		
Normal	Record 2 to 1 on a Scheduled fixture.	Completed; winner gets 3 points, loser 0.		
Invalid	Complete a fixture with no result.	FixtureStateException; stays Scheduled.		
Invalid	Record a result on a Completed fixture.	FixtureStateException; points unchanged.		
Normal	Save, restart, reload.	Same records and same standings.		
Invalid	Damaged line in `fixtures.txt`.	DataFileException naming file and line; nothing from that file loaded.		

7. UML Diagram



8. Console Menu Design

The console application uses a hierarchical, menu-driven interface to provide access to all features. Invalid menu choices or inputs prompt the user with a clear message and re-display the prompt without terminating the program.

Main Menu

1. 1. Team & Squad Management
2. 2. Fixtures & Match Operations
3. 3. Ticketing & Stadium Shop
4. 4. Management Reports
5. 5. Save Data & Exit

1. Team Management Submenu

1. 1.1 Register New Team
2. 1.2 Update / Approve / Suspend Team Status
3. 1.3 Search & Display Team Details
4. 1.4 Add Player / Coach to Squad
5. 1.5 Remove Player / Coach from Squad
6. 1.6 Return to Main Menu

2. Fixtures & Match Operations Submenu

1. 2.1 Register Stadium
2. 2.2 Register Match Official
3. 2.3 Create Fixture
4. 2.4 Assign Match Official to Fixture
5. 2.5 Record Match Result & Update Standings
6. 2.6 Return to Main Menu

3. Ticketing & Shop Submenu

1. 3.1 Purchase Match Tickets
2. 3.2 Add / Update Shop Products & Stock
3. 3.3 Process Shop Transaction (Subtotal, Tax, Payment, Change, Receipt)

4. 3.4 Return to Main Menu

4. Management Reports Submenu

1. 4.1 View League Standings Report
2. 4.2 View Fixtures, Attendance & Ticket Revenue Report
3. 4.3 View Shop Sales & Low Stock Report

4.4 Return to Main Menu

****TO DO****

1.Finalize Team Roles

2.Lecturer Confirmation of the minimum player age constant.

Project Repository:

Link:https://github.com/brandonbell2357-wq/Kingston-Premier-Football-League-KPFL-

Repository Visibility:

Public

Main Branch:

main.